

ADR-003: Demonstration Material Taxonomy

Status: Proposed

Date: September 30, 2025

Context

The Contentstack sales engineering team needs to be able to search using keywords or full sentences through all the demonstration materials and return results based on tags set by the creator/poster of the materials. This taxonomy system also needs to be able to sort through the materials using filters such as most recent and other categories.

Key Non-Functional Requirements (NFRs)

- **Performance:** Ensure fast delivery of search results when querying demonstration materials.
- **Interactivity:** Tags visibly rendered beneath each material. Clicking a tag filters results to all materials with that tag (AND/OR logic supported).
- **Relevance:** Results ranked by hybrid scoring.
- **Freshness:** New/updated materials searchable within 5 minutes of publish/update.
- **Security:** Only authorized users see restricted materials.

Options

1. Postgres FTS (+ tags only)

Use Postgres full-text search (tsvector/tsquery) with a normalized tag schema. No semantic search.

2. Meilisearch / Typesense (managed or self-hosted)

Lightweight search engine with facets and typo-tolerance; optional synonyms; no native vector unless extended.

3. OpenSearch (AWS managed)

Full enterprise search with BM25, facets, and k-NN vector search; heavier ops footprint.

4. Supabase Postgres + pgvector (Hybrid)

Keep canonical data in Postgres (Supabase). Use FTS for keywords + **pgvector** for embeddings (OpenAI or similar). Hybrid ranker combines BM25, cosine similarity, and recency. Facets via SQL.

5. Algolia (SaaS)

Excellent relevance/UX and facets; higher recurring cost; additional auth integration; vector is add-on.

Decision

Chosen Option: Postgres Full Text Search (FTS) with normalized tag schema.

Rationale:

- Minimal infrastructure overhead, leverages existing Postgres instance.
- Native SQL integration simplifies deployment and maintenance.
- Meets performance, interactivity, and freshness NFRs for MVP scope.

Consequences

Positive

- Simple to implement and maintain using existing Postgres infrastructure.
- No external service dependencies or added SaaS cost.
- Native SQL queries support flexible filtering (tags, date, author, etc.).
- Easily extended later with `pgvector` if semantic relevance is required.

Negative

- No deep semantic understanding, exact or keyword-based matches only.
- Relevance tuning (rank weighting, stemming) requires manual optimization.

Risks and Mitigations

Risk	Impact	Mitigation
Limited semantic relevance	Medium	Add <code>pgvector</code> and hybrid ranking later (Option 4).
Performance degradation with scale	Medium	Use materialized views or indexes on <code>tsvector</code> columns; partition data if needed.
Tag filtering complexity	Low	Normalize tag schema with join tables; use indexed lookups.
Inconsistent tagging	Medium	Establish contributor tagging guidelines and validation rules in CMS.
Security gaps in material visibility	High	Enforce role-based access control (RBAC) at query level via Supabase policies.

References

- Contentstack CMS: <https://www.contentstack.com>
- Supabase authentication docs: <https://supabase.com/docs>
- Next.js documentation: <https://nextjs.org/>

My contributions toward the capstone group project have been circumstantial but not overwhelming! I have designed the video library and component library content model within content stack's website, and then after our meeting with Eric Crump, I corrected the model where he suggested changes would be appropriate. I am now working on implementing the JavaScript necessary for a dynamic webpage to display all the videos contentstack deems necessary. These contributions were assigned to me, and I have been promptly completing them to a high degree of excellence.

